

Resumen MUY extenso de prácticas de Sistemas Operativos

Prácticas incluidas: procesos, hilos, señales, memoria compartida, semáforos, archivos, mmap, tuberías sin nombre, FIFOs y despliegue con execve.

0. Mapa global de lo que entra

Estas prácticas construyen progresivamente una caja de herramientas POSIX para programar sistemas concurrentes en C. La idea central es entender cómo un programa interactúa con el sistema operativo usando llamadas al sistema, cómo se crean y coordinan procesos e hilos, y cómo se comunican entre sí mediante distintos mecanismos de IPC.

- Práctica de llamadas al sistema y procesos: open, errno, perror, fork, wait, exit, execve y gettimeofday.
- Práctica de minería multiproceso: dividir un espacio de búsqueda de nonces entre procesos, lanzar ejecutables con execve y matar procesos restantes con kill.
- Práctica de hilos: pthread_create, pthread_join, pthread_exit, mutex y paralelización de integración numérica para estimar π .
- Práctica de señales: signal, sigaction, manejadores, SIGINT, SIGUSR1, SIGALRM, kill, pause/alarm/raise y control interactivo de programas.
- Práctica de memoria compartida: shm_open, ftruncate, mmap, munmap, close, shm_unlink y sincronización entre procesos con semáforos POSIX.
- Práctica de productor-consumidor: primero con memoria compartida y semáforos, después evitando espera activa con semáforos contadores.
- Práctica de archivos: open, read, write, close, lseek, stat/fstat y ordenación de registros en disco.
- Práctica de mmap con archivos: proyectar un archivo en memoria para operar directamente sobre sus registros.
- Práctica de pipes: tuberías sin nombre, cierre correcto de extremos, sincronización implícita y cálculo de π mediante dispatcher/workers/gatherer.
- Práctica de FIFOs: tuberías con nombre, procesos sin parentesco, permanencia de workers/gatherers y despliegue automático mediante fork + execve.

1. Filosofía POSIX y llamadas al sistema

POSIX es una interfaz estándar de programación para sistemas tipo UNIX. En estas prácticas se usa C para llamar directamente a funciones del sistema operativo. Aunque cada llamada tiene parámetros distintos, casi todas comparten una filosofía común: hay que incluir las cabeceras adecuadas, comprobar el valor de retorno, tratar errno si hay error, y usar macros descriptivas en lugar de números mágicos.

1.1 Cabeceras, prototipos y macros

Para usar una llamada al sistema, el compilador necesita conocer su prototipo: nombre, tipo de retorno y parámetros. Esto se consigue con archivos de cabecera .h. Además, muchas llamadas reciben flags o constantes que se representan con macros.

Concepto	Qué significa	Ejemplo
Cabecera	Archivo .h que declara funciones, tipos y macros.	#include <unistd.h> para fork()
Prototipo	Firma de la función que permite compilar correctamente.	pid_t fork(void);
Macro	Nombre simbólico para un valor numérico o flag.	O_CREAT, O_RDONLY, SIGINT
Flags combinables	Bits que se pueden combinar con OR de bits.	O_WRONLY O_CREAT

El operador | no es un OR lógico, sino un OR bit a bit. Se usa para activar varios bits dentro de un entero. Por eso flags como O_CREAT y O_WRONLY se pueden combinar en un único parámetro.

```
fd = open(filename, O_WRONLY | O_CREAT, 0660);
```

1.2 Uso del manual

Las páginas man son la fuente principal para saber qué cabecera incluir, cuál es el prototipo, qué flags existen, qué devuelve una función y qué errores puede producir. La notación open(2) indica la página open en la sección 2 del manual, normalmente llamadas al sistema. perror(3) o errno(3) están en la sección 3, biblioteca C.

- SYNOPSIS: cabeceras y prototipo.
- DESCRIPTION: comportamiento y significado de parámetros.
- RETURN VALUE: qué devuelve si todo va bien y qué devuelve si falla.
- ERRORS: valores de errno posibles y su significado.

1.3 Valor de retorno, errno y perror

La regla práctica es: muchas llamadas al sistema devuelven -1 si fallan y dejan en errno la causa concreta. El programa debe comprobar ese retorno antes de usar el resultado. Por ejemplo, open devuelve un descriptor de archivo no negativo si tiene éxito, y -1 si falla.

```
fd = open(argv[1], O_RDONLY);
if (fd == -1)
{
    if (errno == ENOENT)
        printf("File does not exist.\n");
    perror("open");
    return -1;
}
```

perror() evita tener que comparar errno manualmente si solo se quiere mostrar el error. Imprime el texto que se le pasa, dos puntos y una descripción automática basada en errno.

2. Archivos y descriptores

UNIX trata muchos recursos como archivos: archivos reales, terminales, tuberías, FIFOs, memoria compartida, dispositivos, etc. La abstracción clave es el descriptor de archivo, un entero pequeño que identifica un recurso abierto dentro de la tabla de descriptores del proceso.

2.1 open(), read(), write() y close()

open() abre o crea un archivo y devuelve un descriptor. read() transfiere bytes desde el archivo hacia memoria. write() transfiere bytes desde memoria hacia el archivo. close() libera el descriptor. El sistema operativo

mantiene internamente un offset o índice de lectura/escritura que avanza automáticamente tras cada operación.

Llamada	Cabecera	Uso principal	Retorno típico
open()	<fcntl.h>	Abrir/crear un archivo.	fd >= 0 o -1
read()	<unistd.h>	Leer bytes desde fd a un buffer.	bytes leídos, 0 en EOF, -1 error
write()	<unistd.h>	Escribir bytes desde buffer a fd.	bytes escritos o -1
close()	<unistd.h>	Cerrar descriptor.	0 o -1

```
int fd = open("data.bin", O_RDWR | O_CREAT, 0600);
int x = 0x12345678;
write(fd, &x, sizeof(int));
close(fd);
```

Cuando se escriben enteros en binario, el archivo no contiene texto legible. Por eso cat no sirve para interpretarlo; se usa hexdump -C para ver bytes en hexadecimal y su representación ASCII si existe.

2.2 Offset interno y lseek()

Cada descriptor abierto mantiene un desplazamiento interno. Si se lee un int, el offset avanza sizeof(int). La siguiente lectura continúa desde ahí. Para saltar a otra posición se usa lseek(fd, offset, whence).

whence	Referencia	Ejemplo útil
SEEK_SET	Inicio del archivo.	lseek(fd, 0, SEEK_SET) vuelve al principio.
SEEK_CUR	Posición actual.	lseek(fd, -sizeof(int), SEEK_CUR) retrocede un entero.
SEEK_END	Final del archivo.	lseek(fd, -sizeof(int), SEEK_END) coloca el offset en el último entero.

Un detalle importante: SEEK_END apunta justo después del último byte del archivo. Si un archivo tiene N bytes, los bytes válidos van de 0 a N-1, y el offset N es donde se escribiría el siguiente byte si añadimos datos.

2.3 stat(), fstat() y metadatos

La familia stat obtiene metadatos del archivo, como tamaño, permisos, propietario o fechas. stat trabaja con ruta, fstat con descriptor ya abierto y lstat es similar a stat pero trata enlaces simbólicos de forma distinta.

```
struct stat s;
fstat(fd, &s);
printf("The file size is %ld bytes.\n", s.st_size);
```

En los ejercicios se usa fstat para reservar un array del tamaño adecuado y leer un archivo completo de una vez, sin usar lseek para medir el tamaño.

2.4 Ordenación de archivos: filesort_std

La tarea de ordenación trabaja con un archivo binario datos.bin formado por registros struct evaluation. Cada registro contiene identificador, apellidos, nombre, dos notas, media, nombre de foto, tamaño de foto y datos binarios de la foto. La ordenación se hace por avgrade descendente usando bubble sort.

```
struct evaluation{
    char id[16];
    char surname1[32];
```

```
char surname2[32];
char name[32];
float grade1;
float grade2;
float avgrade;
char photofilename[20];
int photosize;
char photodata[16000];
};
```

La función `gradesort(int fd)` debe ordenar el archivo ya abierto usando `read()`, `write()` y `lseek()`, leyendo y escribiendo estructuras de una en una. No está permitido cargar todo el archivo en memoria para ordenarlo, porque el objetivo es practicar acceso directo a registros en disco.

- Calcular cuántos registros hay: tamaño del archivo / `sizeof(struct evaluation)`.
- Recorrer el archivo con dos bucles como bubble sort.
- Para comparar dos registros adyacentes, posicionar con `lseek` y leer registro `i` y registro `i+1`.
- Si `avgrade` del primero es menor que `avgrade` del segundo, intercambiarlos escribiendo ambos de vuelta en sus posiciones originales.
- Contar cada intercambio y devolver el total. El resultado esperado en la práctica es 13989 swaps.
- Comprobar la corrección con `diff -s datos.bin datosord.bin`.

2.5 mmap() aplicado a archivos

La versión `filesort_mmap` usa `mmap` para proyectar el archivo en memoria. La idea es abrir `datos.bin` con `O_RDWR`, obtener su tamaño con `fstat`, mapearlo con `PROT_READ | PROT_WRITE` y `MAP_SHARED`, y tratarlo como un array de `struct evaluation`. Al modificar el array, los cambios se reflejan en el archivo.

```
struct evaluation *v;
v = mmap(NULL, s.st_size, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
/* v[i] es el registro i del archivo */
```

La lógica de bubble sort es la misma, pero ya no hacen falta `read()`, `write()` ni `lseek()` para cada comparación. Por eso la versión `mmap` suele ser mucho más rápida. En el ejemplo del enunciado, `filesort_std` tarda aproximadamente 260 ms y `filesort_mmap` unos 32 ms, aunque el tiempo real depende de la máquina.

3. Procesos: fork, wait, exit y exec

Un proceso es una instancia de un programa en ejecución, con su propio espacio de direcciones, PID, tabla de descriptores, estado, pila, heap y demás recursos. Las prácticas trabajan con creación de procesos, sincronización padre-hijo, terminación y reemplazo de imagen mediante `exec`.

3.1 fork(): crear procesos

`fork()` crea un proceso hijo casi idéntico al padre. No recibe parámetros. La clave es que retorna dos veces: en el padre devuelve el PID del hijo; en el hijo devuelve 0. Si falla devuelve -1.

```
pid_t pid = fork();
if (pid == -1)
    perror("fork");
else if (pid == 0)
    printf("Soy el hijo, PID=%d, padre=%d\n", getpid(), getppid());
```

```
else
    printf("Soy el padre, PID=%d, hijo=%d\n", getpid(), pid);
```

Después del fork no se garantiza el orden de ejecución. El planificador del sistema operativo decide cuándo corre cada proceso. Por eso no se debe asumir que imprimirá primero el padre o el hijo.

Si el padre termina antes que el hijo, el hijo queda huérfano y es adoptado por init/systemd u otro subreaper. Por eso getppid() en el hijo puede acabar mostrando 1 u otro PID si el padre ya murió. Para evitarlo, el padre debe esperar al hijo con wait o waitpid.

3.2 wait() y waitpid()

wait() bloquea al padre hasta que finalice algún proceso hijo. waitpid() permite esperar a un hijo concreto o usar opciones. wait(&status) equivale conceptualmente a waitpid(-1, &status, 0).

Función	Qué espera	Valor devuelto
wait(&status)	Cualquier hijo.	PID del hijo que terminó o -1.
waitpid(pid, &status, 0)	Un hijo concreto.	PID esperado o -1.
waitpid(-1, &status, WNOHANG)	Cualquier hijo, sin bloquear.	0 si ninguno terminó, PID si alguno terminó.

El entero status no es directamente el valor de exit. Contiene información codificada. Para saber si acabó normalmente se usa WIFEXITED(status). Si es cierto, el valor de salida se obtiene con WEXITSTATUS(status). Si acabó por señal o fallo, WIFEXITED será falso y hay que tratarlo como finalización anómala.

```
int status;
pid_t ended = wait(&status);
if (WIFEXITED(status))
    printf("Hijo %d terminó con exit=%d\n", ended, WEXITSTATUS(status));
else
    printf("Hijo %d terminó anormalmente\n", ended);
```

3.3 exit() y zombies

exit(status) termina el proceso y entrega un código de salida al padre. Si el hijo termina pero el padre no hace wait, el hijo queda como zombie: ya no ejecuta código, pero su entrada de estado se conserva para que el padre pueda recogerla. Si el padre muere, el sistema adopta al zombie y lo recoge.

En los ejercicios se usa exit(1) para indicar retirada o error controlado, exit(0) para éxito, y se comprueba WIFEXITED/WEXITSTATUS en el padre.

3.4 execve() y familia exec

execve() no crea un proceso nuevo. Reemplaza la imagen de memoria del proceso actual por otro programa. Si tiene éxito, no retorna; el código posterior a execve solo se ejecuta si ha fallado.

```
char *argv_exec[] = { "/bin/ls", "-l", NULL };
char *env_exec[] = { NULL };
if (execve(argv_exec[0], argv_exec, env_exec) == -1)
{
    perror("execve");
    exit(-1);
}
printf("Esto no aparece si execve funciona.\n");
```

argv debe terminar en NULL. argv[0] suele ser el nombre/ruta del ejecutable. envp también termina en NULL. Si se usa execlp o execvp, el ejecutable se busca en PATH y se hereda el entorno si la variante no contiene e.

Cuando se combina fork + exec, el padre crea un hijo y el hijo reemplaza su imagen con otro programa. En ese caso los PID son distintos porque fork sí crea un proceso nuevo; exec solo cambia qué programa ejecuta ese proceso hijo.

3.5 gettimeofday() para medir tiempos

gettimeofday obtiene una marca temporal con segundos y microsegundos desde el Epoch UNIX. Se usa tomando una marca antes y otra después de una operación, y restando ambas.

```
struct timeval start, end;
gettimeofday(&start, NULL);
/* trabajo */
gettimeofday(&end, NULL);
long elapsed = (end.tv_sec - start.tv_sec) * 1000000 +
              (end.tv_usec - start.tv_usec);
```

3.6 Carrera de procesos

El ejercicio race modela una carrera donde cada participante es un proceso hijo que ejecuta player(). Cada player puede acabar normalmente con exit(0), retirarse con exit(1) o sufrir un accidente provocando una finalización anormal. El proceso padre mide el tiempo desde el inicio, espera a cada hijo con wait y locuta el resultado según status.

- En /*1*/ se hace fork por cada jugador. En el hijo se llama a player(); en el padre se guarda el PID.
- En /*2*/ se espera a que termine cualquier jugador con pid = wait(&status).
- En /*3*/ se comprueba WIFEXITED(status). Si es falso, accidente.
- En /*4*/ se usa WEXITSTATUS(status) para distinguir finish=0 y retired=1.
- El orden de llegada no se fuerza: wait devuelve el hijo que realmente terminó antes.

4. Multiproceso aplicado: foominer y foominer_mp

El bloque foominer usa una idea tipo blockchain: se tiene un bloque de datos y se busca un nonce de 32 bits tal que el hash SHA-256 del bloque cumpla una dificultad, por ejemplo empezar con cierto número de ceros. Como no se puede predecir el hash, se prueba por fuerza bruta.

4.1 foominer básico

foominer recibe cuatro argumentos: archivo de bloque, dificultad, nonce inicial y nonce final. Carga el bloque, prueba nonces en el rango indicado con fooblock_mine y si encuentra solución guarda un archivo .solved.

```
./foominer myblock 23 0 10000000
./fooblockcheck myblock.solved 23
```

Para compilar se necesita libssl-dev y enlazar con -lcrypto porque la biblioteca usa funciones criptográficas de OpenSSL.

```
cc foominer.c foominerlib.c -o foominer -lcrypto
```

4.2 foominer_mp: reparto del espacio de nonces

foominer_mp lanza varios procesos foominer en paralelo. Cada minero recibe un rango distinto del espacio [0, UINT32_MAX]. Cuando uno encuentra solución, el proceso principal mata a los demás con kill(SIGTERM). Si uno termina sin solución, se espera a los restantes. Si uno falla o crashea, se aborta y se terminan los demás.

La estructura struct foominer_data almacena el PID y rango de trabajo de cada minero: pid, nonce_start y nonce_end. La función foominer_distribute_workload reparte rangos de forma incremental.

- /*1*/ crear cada minero con fork. En el hijo llamar a foominer_exec. En el padre guardar miner[i].pid.
- /*2*/ esperar con pid = wait(&status).
- /*3*/ recorrer el array y hacer kill(miner[i].pid, SIGTERM) a los que sigan activos.
- /*4*/ foominer_exec prepara cadenas con sprintf y llama a execve/execl para lanzar ./foominer con blockfile, difficulty, nonce_start y nonce_end.

```
if ((pid = fork()) == 0)
{
    foominer_exec(blockfile, difficulty, &miner[i]);
    exit(-1);
}
miner[i].pid = pid;
```

Un detalle peligroso: WEXITSTATUS solo conserva 8 bits sin signo. Usar case -1 no es una buena idea conceptual, porque exit(-1) se observa normalmente como 255. Para seguir el enunciado se puede dejar si el material lo espera, pero en código robusto se trataría 255 o EXIT_FAILURE.

5. Hilos POSIX: pthreads

Un hilo es un camino de ejecución dentro de un proceso. A diferencia de los procesos, los hilos comparten memoria global, heap, archivos abiertos y otros recursos. Esto hace que comunicar datos entre hilos sea muy sencillo, pero introduce condiciones de carrera si varios hilos modifican datos compartidos sin sincronización.

5.1 pthread_create

pthread_create crea un hilo y le indica una función de entrada. Esa función debe tener prototipo void *func(void *arg). El argumento es void * para permitir pasar cualquier tipo por referencia, pero dentro del hilo hay que convertirlo al tipo real.

```
void *thread_func(void *arg)
{
    char *msg = (char *)arg;
    puts(msg);
    pthread_exit(NULL);
}

pthread_t th;
pthread_create(&th, NULL, thread_func, "Hola");
```

5.2 pthread_exit y pthread_join

pthread_exit termina el hilo actual y opcionalmente devuelve un puntero. pthread_join espera a que un hilo concreto termine. A diferencia de wait, no existe join de cualquier hilo; hay que indicar el pthread_t concreto.

No hay relación padre-hijo entre hilos: cualquier hilo puede esperar a otro si conoce su identificador. Además, si el hilo principal hace `pthread_exit`, los demás hilos pueden seguir ejecutándose; si `main` retorna, normalmente termina todo el proceso.

5.3 Pasar estructuras a hilos

Cuando un hilo necesita varios datos, se crea una estructura y se pasa su dirección. En la práctica se usa `struct thread_data` con `nrep` y `msg` para que cada hilo imprima una cadena distinto número de veces.

```
struct thread_data {
    int nrep;
    char msg[10];
};

void *thread_func(void *arg)
{
    struct thread_data *d = (struct thread_data *)arg;
    for (int i = 0; i < d->nrep; i++)
        puts(d->msg);
    return NULL;
}
```

Hay que tener cuidado de no pasar la dirección de una variable local que deje de existir antes de que el hilo la use. En los ejemplos, el array de estructuras vive en `main` hasta después de los `join`, así que es válido.

5.4 Condiciones de carrera y mutex

La expresión `x++` no es atómica. Internamente implica leer `x`, sumar 1 y escribir el resultado. Si dos hilos lo hacen a la vez, pueden leer el mismo valor y perder incrementos. Por eso el ejemplo con `TOTAL_COUNT` da resultados incorrectos al usar varios hilos.

Un mutex protege una sección crítica. Solo un hilo puede poseerlo a la vez. Se inicializa con `PTHREAD_MUTEX_INITIALIZER` o `pthread_mutex_init`, se adquiere con `pthread_mutex_lock` y se libera con `pthread_mutex_unlock`.

```
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

pthread_mutex_lock(&mutex);
x++;
pthread_mutex_unlock(&mutex);
```

Aunque proteger cada `x++` corrige el resultado, es muy lento: el programa pasa muchísimo tiempo bloqueando y desbloqueando el mutex. La solución eficiente es que cada hilo acumule localmente y entre a la sección crítica solo una vez para sumar su resultado parcial a la variable global.

```
long local = 0;
for (long i = 0; i < count; i++)
    local++;
pthread_mutex_lock(&mutex);
x += local;
pthread_mutex_unlock(&mutex);
```

6. Integración numérica y cálculo de π con hilos

La práctica de integración calcula integrales usando la regla del trapecio. La idea es aproximar el área bajo una curva dividiendo el intervalo en muchos subintervalos y sumando áreas de trapecios. Cuantos más subintervalos, menor error.

6.1 Regla del trapecio

Para un intervalo $[a,b]$, el área bajo una curva se aproxima con un trapecio cuya base es $b-a$ y cuyas alturas son $f(a)$ y $f(b)$. En muchos pasos, se suma ese cálculo para cada subintervalo.

```
area_trapecio = (x2 - x1) * (f(x1) + f(x2)) / 2
```

La función `tr_integrate(double *result, double x1, double x2, long nstep, double(*f)(double))` integra cualquier función continua con prototipo `double f(double)`. El puntero a función permite pasar `exp`, `sqrt`, una función propia `my_f` o las funciones `f1/f2` para π .

6.2 Funciones usadas para π

Se usan dos funciones cuya integral de 0 a 1 vale $\pi/4$. Por tanto, si la integral aproximada es I , el valor aproximado de π es $4I$.

Función	Interpretación	Integral en $[0,1]$
$\sqrt{1-x^2}$	Cuarto de círculo unidad.	$\pi/4$
$1/(1+x^2)$	Integral $\arctan(x)$.	$\arctan(1)-\arctan(0)=\pi/4$

6.3 Versión multihilo

`tr_integrate_mt` reparte el intervalo total entre `nthreads`. Cada hilo recibe un struct `th_work` con `x_start`, `x_end`, `n_steps`, puntero a función, puntero al resultado compartido y puntero al mutex. El hilo calcula su integral parcial con `tr_integrate` y suma el resultado parcial bajo mutex.

- Crear arrays `pthread_t th[nthreads]` y `struct th_work work[nthreads]`.
- Inicializar un mutex compartido.
- Distribuir el trabajo con `th_distribute_work`: intervalos y pasos para cada hilo.
- Rellenar `f`, `presult` y `pmutex` en cada estructura.
- Crear todos los hilos.
- Hacer `pthread_join` de todos.
- Destruir el mutex si se inicializó dinámicamente.

La suma al resultado global debe ser sección crítica, pero solo se hace una vez por hilo. Eso evita el problema de bloquear millones de veces.

6.4 Interpretación de time

El comando `time` muestra normalmente `real`, `user` y `sys`. `real` es tiempo de reloj: lo que tarda desde fuera. `user` es CPU gastada en código de usuario. `sys` es CPU gastada dentro del kernel. Con varios cores, `real` puede bajar porque se ejecuta en paralelo, mientras `user` puede mantenerse parecido o incluso aumentar porque suma CPU de todos los hilos.

7. Señales POSIX

Las señales son notificaciones asíncronas que el sistema operativo entrega a un proceso. Pueden venir del teclado, del sistema, de otro proceso o del propio proceso. Al recibir una señal, el proceso interrumpe lo que estaba haciendo y ejecuta un manejador, salvo que la acción por defecto sea terminar, ignorar, detener, continuar o volcar core.

7.1 Señales importantes

Señal	Origen típico	Acción por defecto	Uso en prácticas
SIGINT	Ctrl+C o kill -SIGINT.	Terminar.	Primera pulsación muestra estado; segunda termina.
SIGKILL	kill -9.	Terminar inmediatamente.	No se puede capturar ni ignorar.
SIGSEGV	Acceso inválido a memoria.	Core/terminar.	Accidentes en player o fallos anómalos.
SIGALRM	alarm().	Terminar.	Timeout de pinc y cancelación de terminación.
SIGTERM	Solicitud de terminación.	Terminar.	Matar mineros restantes.
SIGCHLD	Hijo terminado.	Ignorar.	Aviso natural cuando un hijo acaba.
SIGSTOP	Ctrl+Z o kill -SIGSTOP.	Detener.	Control de trabajos. No capturable.
SIGCONT	fg/bg o kill -SIGCONT.	Continuar.	Reanudar trabajo detenido.
SIGUSR1	Señal de usuario.	Terminar por defecto.	Solicitar estado parcial.

7.2 signal()

signal(signalnum, handler) registra una función simple que recibe el número de señal y devuelve void. No envía señales: solo cambia el manejador asociado. Se puede restaurar el comportamiento por defecto con SIG_DFL o ignorar con SIG_IGN.

```
void handler(int signalnum)
{
    printf("Señal recibida: %d\n", signalnum);
    signal(SIGINT, SIG_DFL);
}

signal(SIGINT, handler);
```

En el ejemplo de primos, el primer Ctrl+C ejecuta el manejador y restaura SIG_DFL. El segundo Ctrl+C ya ejecuta el comportamiento por defecto y termina el proceso. Si no se restaurase SIG_DFL, cada Ctrl+C ejecutaría el manejador y el programa seguiría.

7.3 sigaction() básico y avanzado

sigaction es la forma recomendada y más flexible. Permite configurar máscara, flags y un manejador avanzado con información extra. En modo simple se rellena sa_handler, se vacía sa_mask con sigemptyset y se ponen flags a 0.

```
struct sigaction a;
a.sa_handler = mysighandler;
sigemptyset(&a.sa_mask);
a.sa_flags = 0;
sigaction(SIGINT, &a, NULL);
```

En modo avanzado se usa `sa_sigaction` y el flag `SA_SIGINFO`. El manejador recibe `signum`, `siginfo_t *info` y `void *context`. `info` permite saber, entre otras cosas, el PID del proceso que envió la señal mediante `info->si_pid`.

```
void handler(int n, siginfo_t *info, void *context)
{
    printf("Signal %d from PID %d\n", n, info->si_pid);
}
```

```
struct sigaction a;
a.sa_sigaction = handler;
sigemptyset(&a.sa_mask);
a.sa_flags = SA_SIGINFO;
sigaction(SIGUSR1, &a, NULL);
```

7.4 kill(), Ctrl+Z, jobs, fg y bg

`kill(pid, sig)` pide al sistema que envíe una señal a un proceso. Desde consola se usa `kill -SIGINT PID`, `kill -SIGSTOP PID`, `kill -SIGCONT PID`, etc. `Ctrl+Z` detiene el proceso en primer plano con `SIGSTOP` y devuelve el control a la shell. `jobs` lista trabajos, `fg` los trae al foreground y `bg` los continúa en background.

Si un proceso está en background, `Ctrl+C` en la terminal no necesariamente lo mata, porque `Ctrl+C` afecta al proceso en primer plano asociado a la terminal. Para terminarlo se puede usar `fg` y luego `Ctrl+C`, o `kill/killall`.

7.5 alarm(), pause(), raise() y llamadas interrumpidas

`alarm(n)` programa una `SIGALRM` dentro de `n` segundos. `pause()` duerme hasta recibir una señal. `raise(sig)` envía una señal al propio proceso. Cuando una señal llega mientras el proceso está bloqueado en una llamada al sistema como `sleep/read/write`, esa llamada puede interrumpirse y retornar antes de tiempo con `EINTR`. `SA_RESTART` puede reiniciar algunas llamadas, pero no todas; `sleep` no se reinicia automáticamente en este contexto.

8. Señales aplicadas: pimc y foominer_signals

8.1 Montecarlo para estimar π

El programa `pimc` genera puntos aleatorios dentro de un cuadrado de lado 1 centrado en el círculo de radio 0.5. Si $x_{\text{delta}}^2 + y_{\text{delta}}^2 \leq 0.25$, el punto cae dentro del círculo. La proporción $\text{in}/(\text{in}+\text{out})$ aproxima $\pi/4$, así que $\pi \approx 4 * \text{proporción}$.

- `pimc.c` básico: recibe número de iteraciones y al final imprime valor calculado.
- `pimc1.c`: añade `timeout` en segundos con `alarm(SIGALRM)`. Si expira, imprime resultado parcial y cuántas iteraciones realizó.
- `pimc2.c`: añade `Ctrl+C` con `SIGINT` para finalizar e imprimir resultado parcial por interrupción.
- `pimc3.c`: añade `SIGUSR1` para imprimir resultado parcial y PID del emisor sin terminar el programa.

La dificultad conceptual está en que el manejador necesita acceder al estado actual: iteración actual, in/out y quizá niter. Para eso se usan variables globales o volatile sig_atomic_t para flags. Lo más limpio es que el manejador marque una bandera y el bucle principal imprima en un punto seguro, porque printf no es async-signal-safe. En prácticas sencillas se suele imprimir directamente desde el manejador, pero conviene saber que no es lo ideal en producción.

8.2 foominer controlado con señales

La ampliación foominer_signals añade control interactivo al minero. La biblioteca tiene una variable global extern hashes que cuenta hashes realizados. Al recibir SIGUSR1, el programa imprime el PID del minero y cuántos hashes lleva comprobados, y continúa.

```
Miner (16098) working, 24561101 hashes checked.
```

Después se modifica SIGINT: la primera pulsación de Ctrl+C no termina el proceso, sino que solicita estado. Para evitar duplicar código, el manejador de SIGINT puede hacer raise(SIGUSR1), es decir, elevar una señal SIGUSR1 al propio proceso.

La última versión añade una ventana de confirmación de 2 segundos: primer Ctrl+C muestra estado y avisa “Press Ctrl+C again within 2 seconds to terminate.” Si llega otro Ctrl+C antes de 2 segundos, termina. Si pasan 2 segundos sin segundo Ctrl+C, una SIGALRM imprime “Termination cancelled.” y se cancela la intención de terminar.

- Estado normal: waiting_for_second_ctrlc = 0.
- Primer SIGINT: imprimir estado, poner waiting_for_second_ctrlc = 1 y alarm(2).
- Segundo SIGINT antes de alarma: imprimir “Termination requested.” y exit.
- SIGALRM: poner waiting_for_second_ctrlc = 0 e imprimir “Termination cancelled.”
- SIGUSR1: imprimir hashes comprobados.

9. Memoria compartida POSIX

La memoria compartida permite que procesos distintos proyecten la misma región de memoria en sus espacios de direcciones. Como los procesos no comparten memoria por defecto, este es un mecanismo IPC explícito. Una región de memoria compartida POSIX se crea/abre con shm_open, se dimensiona con ftruncate y se proyecta con mmap.

9.1 Pasos principales

1. shm_open(name, flags, mode): abrir o crear el objeto. El nombre debe empezar por /.
2. ftruncate(fd, size): establecer tamaño si se acaba de crear.
3. mmap(NULL, size, prot, MAP_SHARED, fd, 0): proyectar en memoria.
4. Usar el puntero devuelto como memoria normal.
5. munmap(ptr, size): desproyectar.
6. close(fd): cerrar descriptor.
7. shm_unlink(name): eliminar el objeto persistente cuando ya no se necesite.

```
fd = shm_open("/my_shm", O_CREAT | O_RDWR, 0600);
ftruncate(fd, sizeof(struct data));
ptr = mmap(NULL, sizeof(struct data), PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
```

En Linux los objetos POSIX de memoria compartida aparecen en /dev/shm. Son persistentes: no desaparecen solo porque termine el proceso. Se eliminan con shm_unlink o borrando el objeto en /dev/shm.

9.2 Lectura y escritura compartida

El primer ejemplo crea /my_shm, lo dimensiona a 80 bytes y escribe “Hello, world!!”. Otro programa puede abrirlo en solo lectura con O_RDONLY, mapearlo con PROT_READ y mostrar la cadena. Otro puede abrirlo con O_RDWR y PROT_WRITE para modificarla. Si un lector imprime cada segundo y un escritor cambia la cadena desde otra consola, se observa la comunicación entre procesos.

Aunque parezca sencillo, puede haber condiciones de carrera: un proceso podría leer mientras otro escribe una cadena y ver datos intermedios o corruptos. Para datos compartidos modificables hace falta sincronización.

10. Semáforos POSIX y sincronización entre procesos

Los semáforos POSIX sirven tanto para hilos como para procesos. En estas prácticas se usan semáforos sin nombre colocados dentro de una región de memoria compartida. Un semáforo de exclusión mutua se inicializa a 1; sem_wait lo decrementa o bloquea si vale 0; sem_post lo incrementa y puede despertar a otro proceso.

```
#include <semaphore.h>
sem_t sem;
sem_init(&sem, 1, 1); /* pshared=1: compartido entre procesos */
sem_wait(&sem);
/* sección crítica */
sem_post(&sem);
```

10.1 Sistema de tickets

El ejemplo tickets tiene una estructura compartida con ntickets, soldtickets, tprice y cash. tickets_init crea e inicializa la memoria compartida. tickets_sell vende tickets hasta que no quedan. Sin sincronización, varios vendedores pueden leer simultáneamente ntickets > 0 y vender más entradas de las existentes.

La solución añade sem_t sem dentro de struct ticketsys y lo inicializa con sem_init(&t->sem, 1, 1). La sección crítica mínima debe proteger el acceso a ntickets al vender cada ticket. Además, las actualizaciones finales de soldtickets y cash también modifican variables compartidas y deben protegerse.

```
sem_wait(&t->sem);
if (t->ntickets > 0)
{
    t->ntickets--;
    sold++;
    keep_selling = 1;
}
else
    keep_selling = 0;
sem_post(&t->sem);
```

No conviene meter todo el bucle de venta dentro del semáforo, porque entonces solo vendería un proceso a la vez durante toda la ejecución. Eso eliminaría la carrera, pero también eliminaría el beneficio de tener varios vendedores. La sección crítica debe ser lo más corta posible.

11. Productor-consumidor con memoria compartida

El modelo productor-consumidor usa un buffer circular compartido. El productor genera items y los inserta en `buffer[in]`, avanzando `in` módulo `BUFFER_SIZE`. El consumidor lee `buffer[out]`, avanzando `out`. `count` indica cuántos elementos hay.

11.1 Versión con espera activa y carreras

La versión inicial usa `while(count == BUFFER_SIZE);` en el productor y `while(count == 0);` en el consumidor. Esto es espera activa: el proceso consume CPU comprobando una condición sin dormir. Además, `count`, `in`, `out` y `buffer` se modifican concurrentemente sin protección, por lo que aparecen datos corruptos.

En el ejercicio se ponen `PROD_DELAY_US` y `CONS_DELAY_US` a 0 para que los errores aparezcan casi inmediatamente. El consumidor detecta corrupción porque espera números consecutivos y muestra el estado del sistema cuando el dato recibido no coincide.

11.2 Primera solución: mutex con semáforo

Para evitar la condición de carrera se añade un semáforo mutex en la estructura compartida y se protege la sección crítica de acceso a `count`, `in/out` y `buffer`. Esto corrige la corrupción, pero si se mantiene la espera activa todavía se puede consumir mucha CPU cuando el buffer está lleno o vacío.

11.3 Segunda solución: semáforos contadores empty/full

La solución completa usa tres semáforos: mutex para exclusión mutua, `empty` para contar huecos libres y `full` para contar elementos disponibles. Inicialmente `empty=BUFFER_SIZE`, `full=0` y `mutex=1`.

```
/* Productor */
newdata = produce_item();
sem_wait(&d->empty);
sem_wait(&d->mutex);
d->buffer[d->in] = newdata;
d->in = (d->in + 1) % BUFFER_SIZE;
sem_post(&d->mutex);
sem_post(&d->full);

/* Consumidor */
sem_wait(&d->full);
sem_wait(&d->mutex);
thedata = d->buffer[d->out];
d->out = (d->out + 1) % BUFFER_SIZE;
sem_post(&d->mutex);
sem_post(&d->empty);
consume_item(thedata);
```

Con esta solución no hay espera activa: si no hay huecos, el productor duerme en `sem_wait(empty)`; si no hay datos, el consumidor duerme en `sem_wait(full)`. Además, la sección crítica queda limitada al acceso al buffer y sus índices.

12. Tuberías sin nombre: pipe()

Una tubería es un canal unidireccional de bytes. Tiene un extremo de lectura y otro de escritura. Se usa `read()` y `write()` igual que con archivos, pero el descriptor no corresponde a un archivo real sino al extremo de la

tubería. Las tuberías sincronizan automáticamente: leer de una tubería vacía bloquea; escribir en una tubería llena bloquea.

12.1 Creación y herencia tras fork

pipe(pipefd) crea dos descriptors: pipefd[0] para leer y pipefd[1] para escribir. Para que otro proceso los conozca, normalmente se crea la tubería antes de fork. El hijo hereda los descriptors abiertos del padre.

```
int pipefd[2];
pipe(pipefd);
if (fork() != 0)
{
    close(pipefd[1]); /* padre lee */
    read(pipefd[0], &n, sizeof(int));
}
else
{
    close(pipefd[0]); /* hijo escribe */
    write(pipefd[1], &n, sizeof(int));
}
```

Es obligatorio cerrar los extremos que no se usan. Si el proceso lector conserva abierto un descriptor de escritura, read puede quedarse bloqueado esperando EOF, porque desde el punto de vista del kernel todavía hay un extremo de escritura abierto. Este es uno de los errores más típicos.

12.2 Comportamiento EOF y SIGPIPE

Si todos los extremos de escritura están cerrados, read retorna los bytes disponibles y luego 0, indicando EOF. Si se escribe en una tubería sin lectores, el escritor recibe SIGPIPE y normalmente termina; si se ignora o maneja SIGPIPE, write falla con -1 y errno=EPIPE.

13. Cálculo de π con procesos y pipes

La práctica picalc_pipes reemplaza la versión multihilo de integración por una arquitectura de procesos comunicados con tuberías sin nombre. Se usan tres tipos de proceso: dispatcher, worker y gatherer. El objetivo es calcular π integrando $f(x)=\sqrt{1-x^2}$ en $[0,1]$ y multiplicando por 4.

13.1 Arquitectura dispatcher / workers / gatherer

Proceso	Responsabilidad	Tubería que usa
dispatcher	Divide el trabajo total en paquetes struct work y los escribe.	Escribe en work pipe.
worker	Lee paquetes, integra cada intervalo y escribe un double resultado.	Lee work pipe, escribe result pipe.
gatherer	Lee doubles, acumula suma total e imprime π .	Lee result pipe.

```
struct work {
    double x_start;
    double x_end;
    long n_steps;
};
```

El programa principal crea dos tuberías: work y results. Luego crea un dispatcher, varios workers y un gatherer con fork. Cada proceso cierra los extremos que no necesita, ejecuta su función correspondiente y termina. El padre cierra sus descriptores y espera a todos los hijos para que la salida aparezca antes de volver a la shell.

- dispatcher debe cerrar lectura de work y ambos extremos de result; solo escribe trabajos.
- worker debe leer de work y escribir en result; cierra work write y result read.
- gatherer debe cerrar work completo y result write; solo lee resultados.
- El padre debe cerrar todos los extremos una vez creados los hijos, o puede impedir EOF.

14. FIFOs: tuberías con nombre

Las FIFOs son tuberías con nombre. A diferencia de pipe(), no hace falta parentesco entre procesos porque la tubería se identifica mediante una ruta en el sistema de archivos. Se crea con mkfifo y se abre con open. Su comportamiento de lectura/escritura es el de una tubería, no el de un archivo normal.

14.1 mkfifo y open bloqueante

mkfifo("myfifo", 0600) crea un archivo especial de tipo p. open("myfifo", O_RDONLY) abre el extremo de lectura y puede bloquear hasta que alguien abra el extremo de escritura. open("myfifo", O_WRONLY) puede bloquear hasta que haya lector. Si se abre con O_RDWR, el proceso abre ambos extremos y puede cambiar el comportamiento de EOF, igual que dejar extremos duplicados abiertos en pipe.

```
mkfifo("work", 0600);
int fdw = open("work", O_WRONLY);
int fdr = open("work", O_RDONLY);
```

Puede haber varios escritores o varios lectores en una FIFO. Los bytes se sincronizan por el kernel. Si hay varios lectores, cada dato será leído por uno de ellos, no por todos; la FIFO reparte el flujo entre lectores según planificación.

14.2 Productor-consumidor con FIFOs

El productor-consumidor con tuberías queda muy simplificado: el productor genera enteros y los escribe en la FIFO; el consumidor lee enteros y verifica la secuencia. No hace falta implementar buffer circular, count, semáforos ni espera activa, porque la tubería ya proporciona buffer interno y bloqueo cuando toca.

15. picalc con FIFOs y despliegue con execve

15.1 picalc_fifos como programas independientes

La versión con FIFOs separa el sistema en tres ejecutables: picalc_dispatcher, picalc_worker y picalc_gatherer. Se crean manualmente los FIFOs work y result y se lanzan procesos desde la shell, normalmente dejando dispatcher y workers en background y gatherer en foreground.

```
mkfifo work
mkfifo result
./picalc_dispatcher 1000000000 50000 work &
./picalc_worker work result &
./picalc_worker work result &
./picalc_gatherer result
```

worker y gatherer aceptan un argumento opcional p de permanente. Si se pasa, no terminan tras un cálculo: cierran y reabren los FIFOs para quedar listos para otro dispatcher. Esto permite reutilizarlos para varios cálculos.

15.2 picalc_fifos: despliegue automático

La última tarea automatiza lo anterior. El ejecutable picalc_fifos recibe los mismos argumentos que picalc_pipes: divisiones, paquetes y número de workers. Crea los FIFOs con mkfifo, hace fork para dispatcher, gatherer y workers, y en cada hijo ejecuta el programa correspondiente con execve. El padre espera a que terminen.

- Preparar argumentos como cadenas: divisiones, paquetes, nombres de FIFOs.
- Crear FIFO work y FIFO result.
- fork + execve para picalc_dispatcher con argumentos: programa, ndiv, nwp, work, NULL.
- fork + execve para cada picalc_worker con argumentos: programa, work, result, NULL.
- fork + execve para picalc_gatherer con argumentos: programa, result, NULL.
- wait para recoger a todos los hijos.
- Opcionalmente unlink de FIFOs al terminar para limpiar.

Aunque execve reemplaza la imagen del proceso hijo, mantiene los descriptors abiertos por defecto. En esta versión, como cada programa abre por su cuenta las FIFOs por nombre, no se depende de variables heredadas ni de descriptors previos.

16. Makefiles: patrones que se repiten

Cada práctica pide Makefile. La idea es que make sin argumentos compile los ejecutables necesarios, y clean borre ejecutables, objetos y archivos generados. También pueden añadirse targets utilitarios como datos o run_fifos, pero no deben estar dentro de all si no se quiere que se ejecuten por defecto.

```
CC=cc
CFLAGS=-Wall -Wextra -g

all: programa

programa: main.o lib.o
    $(CC) main.o lib.o -o programa -lm

clean:
    rm -f *.o programa *.solved datos.bin datosord.bin
```

Para OpenSSL se añade -lcrypto al enlazar. Para matemáticas, -lm. Para pthreads, según sistema se usa -pthread tanto al compilar como al enlazar. Para memoria compartida y semáforos en Linux moderno suele bastar con libc, aunque en sistemas antiguos podía hacer falta -lrt o -pthread según el caso.

17. Chuleta rápida de APIs

Tema	Funciones clave	Cabeceras típicas	Idea esencial
Archivos	open/read/write/close/ lseek/stat	<fcntl.h>, <unistd.h>, <sys/stat.h>	Operar con descriptors y offset.
Procesos	fork/wait/waitpid/exit/	<unistd.h>, <sys/wait.h>,	Crear hijos, esperar y

	execve/getpid	<stdlib.h>	ejecutar programas.
Tiempo	gettimeofday	<sys/time.h>	Medir intervalos en microsegundos.
Hilos	pthread_create/join/exit	<pthread.h>	Ejecutar funciones concurrentes compartiendo memoria.
Mutex	pthread_mutex_lock/unlock	<pthread.h>	Proteger secciones críticas entre hilos.
Señales	signal/sigaction/kill/alarm/raise/pause	<signal.h>	Notificaciones asíncronas y control de procesos.
Memoria compartida	shm_open/ftruncate/mmap/munmap/shm_unlink	<sys/mman.h>, <fcntl.h>, <unistd.h>	Compartir regiones de memoria entre procesos.
Semáforos	sem_init/sem_wait/sem_post	<semaphore.h>	Exclusión mutua y sincronización.
Pipes	pipe/read/write/close	<unistd.h>	Canal unidireccional heredado por fork.
FIFOs	mkfifo/open/read/write	<sys/stat.h>, <fcntl.h>, <unistd.h>	Pipe con nombre para procesos no relacionados.

18. Errores típicos que hay que evitar

- No comprobar retornos: open, fork, pipe, mmap, read, write, pthread_create, sem_init, etc. pueden fallar.
- Usar WEXITSTATUS sin comprobar antes WIFEXITED.
- Olvidar cerrar extremos de una tubería, provocando bloqueos porque no llega EOF.
- Abrir una FIFO con O_RDWR cuando solo se quería leer o escribir, alterando el comportamiento de cierre.
- Leer o escribir estructuras con tamaños incorrectos: siempre sizeof(struct tipo), no números mágicos.
- Proteger demasiado código con un semáforo/mutex y destruir el paralelismo.
- Proteger demasiado poco y dejar variables compartidas sin sincronización.
- Usar espera activa cuando puede usarse sem_wait, pipe o bloqueo del sistema operativo.
- Olvidar que execve no retorna si tiene éxito.
- No terminar arrays argv/envp de execve en NULL.
- Pasar a hilos punteros a variables locales que dejan de existir o cambian antes de que el hilo las use.
- No inicializar semáforos compartidos con pshared=1 cuando se usan entre procesos.
- No limpiar memoria compartida persistente con shm_unlink o FIFOs con unlink/rm.
- Usar printf dentro de manejadores de señal como si fuera completamente seguro; en prácticas puede valer, pero en código robusto se prefieren flags.

19. Resumen mental final por bloques

Procesos

fork duplica el proceso; padre e hijo continúan desde el fork con valores de retorno distintos. wait sincroniza y recoge el estado. exit entrega un estado al padre. execve sustituye el programa del proceso actual. Para paralelizar con procesos se reparte trabajo, se lanza cada hijo y se espera a resultados o terminaciones.

Hilos

pthread_create arranca una función dentro del mismo proceso. Los hilos comparten memoria, por eso comunicar es fácil y corromper datos también. pthread_join sincroniza. Mutex protege secciones críticas. La solución eficiente no es bloquear en cada microoperación, sino acumular localmente y sincronizar poco.

Señales

Las señales llegan en cualquier momento. signal es simple; sigaction es robusta y permite saber quién envió la señal. SIGINT, SIGUSR1 y SIGALRM se usan para control interactivo, resultados parciales y timeouts. kill sirve para enviar señales desde procesos o consola.

Memoria compartida

shm_open + ftruncate + mmap permite compartir estructuras entre procesos. Pero compartir memoria no sincroniza nada: hay que usar semáforos. Los semáforos pueden ser mutex o contadores para modelar recursos disponibles, como huecos e ítems en productor-consumidor.

Archivos

read/write operan desde/hacia memoria y avanzan el offset. lseek cambia el offset. stat/fstat da tamaño y metadatos. mmap convierte un archivo en una región de memoria y permite tratar registros como un array, con mucho mejor rendimiento en ciertos casos.

Pipes y FIFOs

Las tuberías son flujo de bytes y sincronizan automáticamente. pipe sirve para procesos relacionados porque los descriptors se heredan por fork. FIFOs tienen nombre y sirven para procesos independientes. En ambos casos, cerrar correctamente extremos es esencial para que se detecte EOF y no haya bloqueos.

20. Cómo estudiarlo para examen/práctica

- Aprende el patrón de cada API: cabecera, llamada, retorno, error y uso típico.
- Dibuja siempre quién lee, quién escribe y qué descriptor usa cada proceso.
- Para concurrencia, identifica datos compartidos y sección crítica antes de escribir semáforos/mutex.
- Para procesos, piensa: ¿quién crea?, ¿quién espera?, ¿qué pasa si falla?, ¿qué recursos quedan abiertos?
- Para señales, separa registrar manejador de enviar señal. signal/sigaction registran; kill/raise/alarm generan.
- Para exec, recuerda: si sale bien, no vuelve. Todo código posterior es manejo de error.
- Para mmap, recuerda: abrir, obtener tamaño, mapear, usar, munmap, close.
- Para pipes/FIFOs, recuerda: flujo unidireccional, read bloquea si vacío, write bloquea si lleno, EOF solo si no queda ningún escritor abierto.

Apéndice A. Plantillas de código útiles

A.1 Patrón fork + wait

```
pid_t pid = fork();
if (pid == -1) {
    perror("fork");
    exit(EXIT_FAILURE);
}
```

```

if (pid == 0) {
    /* código hijo */
    exit(0);
}
int status;
pid_t ended = wait(&status);
if (ended == -1) {
    perror("wait");
    exit(EXIT_FAILURE);
}
if (WIFEXITED(status))
    printf("exit=%d\n", WEXITSTATUS(status));
else
    printf("terminación anormal\n");

```

A.2 Patrón sigaction con SA_SIGINFO

```

static void handler(int sig, siginfo_t *info, void *ctx)
{
    (void)ctx;
    printf("signal=%d sender=%d\n", sig, info->si_pid);
}

struct sigaction a;
a.sa_sigaction = handler;
sigemptyset(&a.sa_mask);
a.sa_flags = SA_SIGINFO;
sigaction(SIGUSR1, &a, NULL);

```

A.3 Patrón memoria compartida + semáforo

```

struct shared {
    int value;
    sem_t mutex;
};

fd = shm_open("/my_shm", O_CREAT | O_RDWR, 0600);
ftruncate(fd, sizeof(struct shared));
s = mmap(NULL, sizeof(struct shared), PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
sem_init(&s->mutex, 1, 1);

sem_wait(&s->mutex);
s->value++;
sem_post(&s->mutex);

```

A.4 Patrón pipe hijo escribe / padre lee

```

int p[2];
pipe(p);
if (fork() == 0) {
    close(p[0]);
    write(p[1], &data, sizeof(data));
    close(p[1]);
    exit(0);
}

```

```
close(p[1]);
while (read(p[0], &data, sizeof(data)) > 0)
    printf("%d\n", data);
close(p[0]);
wait(NULL);
```

A.5 Patrón FIFO

```
mkfifo("work", 0600);
/* escritor */
int w = open("work", O_WRONLY);
write(w, &x, sizeof(x));
close(w);

/* lector */
int r = open("work", O_RDONLY);
while (read(r, &x, sizeof(x)) > 0)
    process(x);
close(r);
```

21. Guía de entregables y carpetas por práctica

Esta sección convierte todo lo anterior en una lista operativa de qué se crea en cada directorio, qué ejecutables deberían salir y qué comportamiento se espera observar. Sirve para revisar rápidamente si la práctica está completa.

21.1 Laboratorios iniciales: llamadas al sistema, procesos y foominer

Carpeta	Archivos/ejecutables	Objetivo
lab4/ejemplos	programas de open, perror, execve y tiempo	Familiarizarse con manual, errores y ejecución de otros programas.
lab5/race	race.c, race.h, player.c, Makefile	Lanzar corredores como procesos y recoger resultados con wait.
lab5/foominer	foominer.c, foominerlib.c/.h, fooblockcheck.c, Makefile	Minado secuencial de un bloque en un rango de nonces.
lab5/foominer_mp	foominer_mp.c y archivos de foominer	Lanzar varios mineros, repartir rangos y terminar los restantes al encontrar solución.

El patrón importante aquí es fork + wait + comprobación de status. Si un hijo ejecuta otro programa, se usa fork + exec. Si hay que parar otros procesos, se usa kill con sus PID guardados.

21.2 Práctica de hilos e integración

Carpeta	Archivos/ejecutables	Objetivo
lab6/ejemplos	thread_*.c, mutex_*.c	Entender pthread_create, pthread_join, argumentos y carreras.
lab6/picalc	integrate.c/.h, f1.c, f2.c, f.h, picalc.c, Makefile	Calcular π con integración secuencial y multihilo.

El punto clave es no confundir hilos con procesos. Los hilos comparten memoria, por eso el resultado global puede estar en una variable compartida; pero para modificarlo correctamente hace falta mutex.

21.3 Práctica de señales

Carpeta	Archivos/ejecutables	Objetivo
lab7/ejemplos	ejemplo1.c ... ejemplo5.c	Registrar manejadores, probar Ctrl+C, SIGUSR1, SIGSTOP, SIGCONT y SIGALRM.
lab7/pimc	pimc.c, pimc1.c, pimc2.c, pimc3.c, Makefile	Montecarlo con timeout, interrupción y resultados parciales por señal.
lab7/foominer_signals	foominer modificado + Makefile	Mostrar progreso con SIGUSR1 y Ctrl+C controlado.

- ejemplo1 usa signal para capturar SIGINT una vez y luego restaurar SIG_DFL.
- ejemplo2 repite la idea con sigaction y sa_handler.
- ejemplo3 usa sigaction avanzado con SA_SIGINFO para obtener info->si_pid.
- ejemplo4 captura SIGINT y SIGUSR1 con el mismo manejador, distinguiendo signum.
- ejemplo5 añade SIGALRM y alarm(10) para finalizar por temporizador.

21.4 Práctica de memoria compartida y semáforos

Carpeta	Archivos/ejecutables	Objetivo
lab8/ejemplos	ejemplo1_create/read/write y ejemplo2_init/sell	Crear memoria compartida, leer/escribir y observar carreras.
lab8/tickets	tickets.h, tickets_init.c, tickets_sell.c, Makefile	Resolver carrera del sistema de tickets con semáforo compartido.

La solución de tickets no debe serializar toda la venta en un único vendedor. Lo correcto es proteger operaciones cortas sobre variables compartidas, permitiendo que varios procesos trabajen y se alternen con seguridad.

21.5 Práctica de productor-consumidor

Carpeta	Archivos/ejecutables	Objetivo
lab9/ejemplos	ejemplo1_prod.c, ejemplo1_cons.c, ejemplo1.h	Reproducir corrupción por carreras en buffer circular.
lab9/prodcons	prodcons.h, prod.c, cons.c, Makefile	Añadir semáforo mutex para evitar carreras.
lab9/prodcons2	copia mejorada	Añadir empty/full para evitar espera activa.

La diferencia entre prodcons y prodcons2 es fundamental: el primero corrige la integridad de los datos, pero puede seguir gastando CPU si mantiene bucles de comprobación; el segundo usa semáforos contadores para dormir cuando no se puede producir o consumir.

21.6 Práctica de archivos, mmap y ordenación

Carpeta	Archivos/ejecutables	Objetivo
lab10/ejemplos	ejemplo1.c, ejemplo2.c, ejemplo1_uchar.c, ejemplo1_add.c, ejemplo2_uchar.c, ejemplo2_readall.c, ejemplo2_mmap.c	Practicar open/read/write/lseek/fstat/mmap.
lab10/filesort	filesort.h, filesort.c, showgrades.c, gradesort_std.c, gradesort_mmap.c, Makefile, datos.tgz	Ordenar datos.bin por avgrade descendente con dos métodos.

El target datos del Makefile debe descomprimir datos.tgz para regenerar datos.bin y datosord.bin. clean puede borrar datos.bin/datosord.bin porque se regeneran. all debe compilar ejecutables, no ejecutar make datos automáticamente.

21.7 Práctica de tuberías y FIFOs

Carpeta	Archivos/ejecutables	Objetivo
lab11/ejemplos	ejemplo1.c	Tubería sin nombre entre hijo y padre.
lab11/picalc_pipes	picalc_pipes.c/h, picalc_agents.c, integrate.c/h, f1.c, f.h, Makefile	Cálculo de π con dispatcher, workers y gatherer usando pipe.
lab12/ejemplos	ejemplo1.c, ejemplo1_send.c, ejemplo1_recv.c	FIFOs, open bloqueante y varios lectores/escritores.
lab12/prodcons_pipes	ejemplo2_prod.c, ejemplo2_cons.c, ejemplo2.h, Makefile	Productor-consumidor con FIFO.
lab12/picalc_fifos	picalc_dispatcher.c, picalc_worker.c, picalc_gatherer.c, picalc_fifos.c, Makefile	Misma arquitectura con FIFOs y luego despliegue con execve.

22. Desarrollo detallado de gradesort_std

La versión estándar de ordenación es una de las tareas más delicadas porque obliga a pensar en el archivo como una secuencia de registros fijos. No se puede cargar todo el archivo, así que cada comparación requiere posicionarse, leer y, si procede, volver atrás para escribir.

8. Abrir datos.bin en filesort.c con O_RDWR. La función gradesort recibe fd ya abierto.
9. Usar fstat(fd, &st) para obtener st.st_size.
10. Calcular nrecords = st.st_size / sizeof(struct evaluation).
11. Bucle externo j desde 0 hasta nrecords-2. En cada pasada, swaps_pass=0.
12. Bucle interno i desde 0 hasta nrecords-2-j.
13. Calcular offset = i * sizeof(struct evaluation).
14. lseek(fd, offset, SEEK_SET).
15. read(fd, &a, sizeof(a)); read(fd, &b, sizeof(b));
16. Si a.avgrade < b.avgrade, volver con lseek(fd, offset, SEEK_SET) y escribir b y luego a.
17. Incrementar contador global de swaps y marcar swaps_pass=1.
18. Si una pasada no hace swaps, cortar porque ya está ordenado.

La comparación es descendente: nota media mayor debe quedar antes. Si se ordenase ascendente por error, diff fallaría aunque el número de registros fuese correcto.

```
if (a.avgrade < b.avgrade)
{
    lseek(fd, offset, SEEK_SET);
    write(fd, &b, sizeof(struct evaluation));
    write(fd, &a, sizeof(struct evaluation));
    swaps++;
    swaps_pass = 1;
}
```

El número esperado de swaps es 13989. Si no coincide, suele deberse a una comparación invertida, a mover mal el offset, a escribir solo un registro o a no reiniciar correctamente el offset tras leer los dos registros.

23. Desarrollo detallado de picalc_pipes

picalc_pipes concentra varios conceptos: pipe, fork, cierre de descriptores, read/write de estructuras y wait. El proceso principal no calcula; solo despliega la arquitectura.

23.1 Descriptores por proceso

Proceso	work[0] lectura	work[1] escritura	result[0] lectura	result[1] escritura
dispatcher	cerrado	abierto	cerrado	cerrado
worker	abierto	cerrado	cerrado	abierto
gatherer	cerrado	cerrado	abierto	cerrado
padre	cerrado tras fork	cerrado tras fork	cerrado tras fork	cerrado tras fork

El cierre en el padre es tan importante como en los hijos. Si el padre conserva work[1] abierto, los workers pueden quedarse esperando más paquetes aunque dispatcher ya haya terminado. Si conserva result[1] abierto, el gatherer puede no recibir EOF.

23.2 Lectura/escritura de estructuras

dispatcher escribe struct work completos en la tubería de trabajo. worker debe leer exactamente sizeof(struct work). Como una tubería es flujo de bytes, en código robusto se implementaría una función read_full para asegurarse de recibir todos los bytes. En prácticas pequeñas suele asumirse que al escribir estructuras pequeñas y leer el mismo tamaño se recibe completo, pero conceptualmente lo correcto es saber que read puede devolver menos bytes.

```
struct work w;
while (read(workpipe, &w, sizeof(w)) > 0)
{
    double partial = integrate(w.x_start, w.x_end, w.n_steps);
    write(resultpipe, &partial, sizeof(partial));
}
```

24. Desarrollo detallado de picalc_fifos

La versión FIFO separa lo que antes era una función en procesos lanzables. Cada ejecutable tiene una responsabilidad mínima.

- picalc_dispatcher: parsea num div, num work packages y nombre del FIFO work. Abre work con O_WRONLY, llama a dispatcher(fd, nwp, total_steps), cierra fd.
- picalc_worker: parsea work fifo, result fifo y opcional p. Abre work con O_RDONLY y result con O_WRONLY, llama a worker(workfd, resultfd), cierra y si p está activo repite.
- picalc_gatherer: parsea result fifo y opcional p. Abre result con O_RDONLY, llama a gatherer(resultfd), cierra y si p está activo repite.
- picalc_fifos: crea FIFOs y lanza los tres tipos anteriores con fork + execve.

El orden de apertura importa porque open en FIFO puede bloquear. Por eso en pruebas se suelen lanzar procesos en background. En el target run_fifos del Makefile se pueden crear los FIFOs, lanzar gatherer y workers en background, y finalmente lanzar dispatcher para iniciar el flujo.

```
run_fifos:
    rm -f work result
    mkfifo work result
    ./picalc_gatherer result &
```

```
./picalc_worker work result &  
./picalc_worker work result &  
./picalc_dispatcher 100000000 1000 work
```

25. Preguntas típicas de comprensión y respuesta corta

25.1 ¿Por qué read devuelve 0?

Porque ha llegado EOF. En archivos, significa fin de archivo. En tuberías/FIFOs, significa que no quedan datos y todos los escritores han cerrado el extremo de escritura.

25.2 ¿Por qué mi pipe se queda bloqueada?

Normalmente porque algún proceso conserva abierto un descriptor de escritura, así que el lector no ve EOF; o porque se intenta leer sin escritores o escribir sin lectores en condiciones que bloquean.

25.3 ¿Por qué una FIFO abierta con O_WRONLY se queda esperando?

Porque no hay ningún lector todavía. open bloquea hasta que el otro extremo se abra. Es comportamiento normal de FIFOs.

25.4 ¿Por qué mmap es más rápido que read/write/lseek?

Porque evita miles de llamadas al sistema y permite que el sistema operativo cargue páginas bajo demanda. El programa opera sobre memoria y el kernel gestiona la coherencia con el archivo.

25.5 ¿Por qué no basta con memoria compartida?

Porque compartir memoria no protege de accesos simultáneos. Si dos procesos modifican las mismas variables sin sincronización, aparecen condiciones de carrera. Hace falta semáforo/mutex.

25.6 ¿Por qué usar semáforos empty y full si ya tengo mutex?

El mutex protege la sección crítica, pero no dice cuándo hay huecos o datos. empty y full sincronizan condiciones: dormir al productor si no hay huecos y dormir al consumidor si no hay datos. Evitan espera activa.

25.7 ¿Qué diferencia hay entre SIGTERM y SIGKILL?

SIGTERM pide terminar y puede capturarse o ignorarse. SIGKILL termina obligatoriamente y no puede capturarse. Por eso se usa SIGTERM para terminación limpia y SIGKILL como último recurso.

25.8 ¿Qué diferencia hay entre signal y sigaction?

signal es simple y menos flexible. sigaction permite flags, máscaras y manejadores avanzados con siginfo_t. Para código serio se prefiere sigaction.

25.9 ¿Qué pasa si execve funciona?

El proceso deja de ejecutar el programa actual y pasa a ejecutar el nuevo. execve no retorna. Si retorna, es que ha fallado.

25.10 ¿Por qué WEXITSTATUS no debe usarse solo?

Porque solo tiene sentido si WIFEXITED(status) es verdadero. Si el proceso terminó por señal, WEXITSTATUS no representa un exit normal.

26. Mini-glosario

Término	Definición práctica
Descriptor de archivo	Entero que identifica un recurso abierto dentro de un proceso.
Offset	Posición actual de lectura/escritura asociada a un descriptor de archivo.
Proceso zombie	Proceso terminado cuyo padre aún no ha recogido su estado con wait.
Huérfano	Proceso cuyo padre terminó; es adoptado por init/systemd.
Sección crítica	Bloque de código que accede a datos compartidos y debe ejecutarse con exclusión mutua.
Condición de carrera	Error dependiente del orden temporal de accesos concurrentes.
Espera activa	Bucle que consume CPU esperando a que cambie una condición.
IPC	Comunicación entre procesos: pipes, FIFOs, memoria compartida, señales, etc.
MAP_SHARED	Los cambios en la proyección son compartidos con otros procesos o con el archivo.
FIFO	Tubería con nombre abierta mediante open.
EOF	Fin de archivo o cierre definitivo del extremo de escritura en una tubería.
SIGPIPE	Señal al escribir en una tubería sin lectores.
SA_SIGINFO	Flag de sigaction que permite recibir siginfo_t con información extra.
pshared=1	sem_init configurado para semáforo compartido entre procesos.